Converge 4A

Automated Appointment Arrangement Application

Project Proposal

Peeranat Thiwongsa 662115034

Supawit Promma 662115049

Bachelor of Science

Software Engineering Program

College of Arts, Media, and Technology

Chiang Mai University

3rd Year

Project Advisor

Apitchaka Singjai, Dr.Techn.

Document History

Version	History	Status	Date	Editable	Reviewer
v0.1.0	- Create Chapter 1,2	C	18-3-2026	PT, SP	-
v0.1.0	- Review Chapter 1,2	R	19-3-2026	-	AS
v0.1.1	- Update Chapter 1	U	22-3-2026	PT, SP	-
v0.1.2	- Update Chapter 2	U	23-3-2026	PT, SP	-
v0.1.3	- Create Chapter 3	C	23-3-2026	PT, SP	-
v0.1.3	- Review Chapter 1,2,3	R	23-3-2026	-	AS
v0.1.4	- Update Chapter 1,2,3	U	24-3-2026	PT,SP	-
v0.1.4	- Review Chapter 1,2,3	R	24-3-2026	-	AS
v1.0.0	- Update Chapter 1,2,3	U	24-3-2026	PT,SP	-
v1.0.1	- Update Chapter 1,2	U	16-4-2026	PT,SP	-
v1.0.2	- Update Chapter 1,2,3	U	17-4-2026	PT,SP	-
v1.0.3	- Update Chapter 1,2,3	U	18-4-2026	PT,SP	-

*PT = Peeranat Thiwongsa
*SP = Supawit Promma
*AS = Apitchaka Singjai, Dr.Techn.

Version:
document name vX.Y.Z
X: External published
Y: Test approval
Z: Internal review

Status:
C = Create
R = Reviewed by advisor
U = Update
D = Delete

TABLE OF CONTENTS

Chapter One | Introduction and Rationale **4**
1.1 Motivation 4
1.2 Problem Statement 4
1.3 Purpose 4
1.4 Target Users 5
1.5 Project Scope 5
1.6 Feature Map 7

Chapter Two | Literature Review **8**
2.1 Business Review / Software Review 8
2.2 Architectural Foundations & Prior Art 10
2.2.1 Constraint Satisfaction Modeling for Complex Timetabling 10
2.2.2 Travel Buffer System 10
2.2.3 Heuristic Search and Interactive Explainable AI (XAI) 11
2.3 Tool and Technology Review 11
2.3.1 Frontend Language 11
2.3.2 Frontend Framework 12
2.3.3 Backend Language 12
2.3.4 Backend Framework 13
2.3.5 Database 13
2.3.6 Architectural Pattern 14
2.3.7 Deployment Architecture 14
2.3.8 Containerization 15

Chapter Three | Project Plan **16**
3.1 Objectives 16
3.2 Type of System Users and the Users' Characteristics 16
3.3 Acronyms and Definitions 17
3.4 System Architecture 18
3.5 Use Case Diagram 19
3.6 Product Features 20
3.6.1 Automated Data Ingestion Form 20
3.6.2 Constraint Satisfaction AI Engine 20
3.6.3 Travel Buffer System 20
3.6.4 Branch Capacity Monitoring 21
3.6.5 Interactive XAI Decision Dashboard 21
3.6.6 Automated Disaster Recovery 22
3.7 SDLC 22
3.8 Limits 22
3.9 Milestones 24

Chapter Four | References **28**

Chapter One | Introduction and Rationale

1.1 Motivation

Multi-branch tutoring centers operate in a highly dynamic environment where instructors frequently rotate between different branches to meet student demand. For these institutions, efficient scheduling is the operational backbone however, as the number of students, teachers, and physical branches grows, the complexity of coordinating weekly classes scales exponentially. Currently, administrators rely on manual systems to manage the schedule.

The booking process begins with a customer selecting a subject, followed by choosing a preferred instructor. The administrator must then manually filter through a static Excel spreadsheet to find and present that specific instructor's available time slots. If none of those times work for the customer, the administrator must repeat the entire filtering process for alternative instructors until a suitable time is found. Once a time is finally selected, the administrator acts as a middleman, sending a minimum of three separate LINE messages: first to the instructor to verify the requested slot, then back to the customer to finalize the booking, and finally to a staff group chat, So the class can be manually logged.

When an instructor is booked for consecutive classes at different branches, administrators must manually calculate and block out their standard 30-minute commute buffer. Managing these transition times by hand across dozens of instructors significantly slows down the booking process. Furthermore, without a branch capacity tracking system, branches are prone to overbooking, leading to situations where teachers and students arrive at a branch only to discover that there is no physical space available.

Ultimately, this repetitive workflow consumes an estimated 1 to 2 hours of administrative time each day and introduces significant human error into the weekly recurring schedule, and forces administrators to manually cross-reference multiple tables within a massive, cluttered spreadsheet just to find a single available teacher for a requested branch.

1.2 Problem Statement

Administrators at multi-branch tutoring schools need an automated, commute-aware scheduling system because the current manual workflow wastes up to 2 hours of administrative time daily. The process requires sending at least three messages per booking, relies on a cluttered spreadsheet that obscures operational insights, and frequently results in branch overbooking. To resolve this, the administration requires an algorithmic system capable of automatically evaluating inter-branch travel times and making sure schedules is optimal.

1.3 Purpose

The primary purpose of the Converge application is to develop an automated scheduling and resource management system that eliminates the operational bottlenecks and inefficiency associated with manual booking in multi-branch tutoring centers. By replacing fragmented spreadsheet workflows and repetitive messaging with an intelligent, commute-aware engine, this

project aims to provide administrators with a centralized platform to seamlessly align student requests, instructor availability, and physical branch capacity in real-time.

1.4 Target Users

The target users of Converge are categorized into primary users, who interact directly with the scheduling interface, and secondary users, representing the organizations that benefit from the system's efficiency.

Primary Target Users

1. Administrator are the primary operators who interact with the system daily to manage incoming requests from parents and input bookings into the interface. Their main objective is to finalize complex, multi-branch schedules in a matter of seconds while the customer is still present, ensuring a smooth and highly reliable booking process. By using an automated system, they aim to eliminate the professional anxiety associated with manual errors, such as overbooking a branch or taking an excessively long time to find an available slot, which can damage the school's reputation

Secondary Target Users

1. Instructors are the secondary users who interact with the system's external data-ingestion form. Their primary objective is to quickly and easily submit their weekly availability using a structured interface, rather than typing unstructured paragraphs in a chat application. Furthermore, they are the primary beneficiaries of the system's routing engine, relying on its accuracy to ensure they are protected from double-booked and are always provided with realistic, physically possible commute buffers between branch locations.

1.5 Project Scope

This project focuses on the design and development of Converge, an enterprise-grade scheduling and resource management system utilizing heuristic optimization for multi-branch tutoring centers. The architecture replaces manual, error-prone workflows with an automated decision-support pipeline:

1. **Automated Data Ingestion Form:** An account-less, structured web form replacing unstructured manual schedule entry, ensuring all instructor availability is normalized into discrete binary matrices before entering the database.
2. **Constraint Satisfaction AI Engine:** A backend algorithm that evaluates multiple requirements (instructor availability, branch capacity) to guarantee valid, conflict-free schedules while prioritizing administrative preferences.
3. **Travel Buffer System:** A configuration-driven module that utilizes a customizable, admin-defined travel duration matrix between branches. This automatically injects physical commute buffers, instantly pruning the combinatorial search space without relying on external geographic APIs.

4. **Branch Capacity Monitoring:** Monitors concurrent bookings against the fixed physical capacity of each branch, explicitly flagging full time slots to prevent overbooking and ensuring students are never left without physical space.
5. **Interactive XAI Decision Dashboard:** A centralized web interface that exposes the heuristic scoring of the backend engine, allowing staff to rapidly understand algorithmic decisions and manually negotiate alternative times with clients based on clear logistical data.
6. **Automated Disaster Recovery:** Scheduled, asynchronous database snapshot generation to ensure business continuity and data sovereignty for the B2B client.

Out of Scope

1. **Self-Service Client Booking Portal:** The client explicitly requires a manual, administrative-driven sales funnel to negotiate soft constraints and upsell packages to parents. Furthermore, following established research on Interactive AI in timetabling[1], the system acts as an administrative decision-support tool rather than a fully automated "black-box" consumer app.
2. **Payment Gateway and Financial Processing:** The client currently utilizes a proprietary, closed-source application for financial processing, which operates on a time-token ledger (where clients pre-purchase hours). Because this system lacks APIs for real-time balance inquiries and token deduction, Converge is architected strictly as a decoupled routing engine. Attempting to build fragile data-scraping workarounds to force state synchronization with a closed-source ledger introduces severe security risks—such as double-spending tokens or ledger corruption and is therefore strictly out of scope.
3. **Probabilistic Machine Learning / Predictive AI:** The system utilizes deterministic Symbolic AI (Constraint Logic Programming) to guarantee mathematically exact schedules. It will not utilize statistical or generative AI (e.g., Large Language Models or predictive trend analysis), as guessing schedules introduces unacceptable logistical risks for physical resource allocation.

1.6 Feature Map

Pain Point	Feature Description	Codename	Success Metric
Teachers submit availability via text paragraphs, admins need to manually interpret and enter data.	An account-less, structured web form where teachers select their identity to submit availability, instantly normalizing the data into the database.	Data-Ingest Form	Drastically reduce manual data entry required for instructor availability.
Manual booking workflows waste 1-2 hours daily on complex spreadsheet cross-referencing.	Evaluates instructor, room, and time variables as a formal CSP to ensure valid conflict-free schedules.	Constraint Satisfaction AI Engine	Reduce average booking time to under 2 minutes.
Manual transit calculation is highly vulnerable to human error.	Uses a configurable travel matrix to automatically inject physical commute buffers between branches.	Auto-Buffer	Automate transit enforcement to prevent human errors caused by missed travel buffers.
Lack of capacity tracking leads to overbooking a branch, leaving teachers and students without a physical space upon arrival.	Monitors concurrent bookings against the fixed physical capacity of each branch, flagging full time slots to prevent overbooking.	Capacity-Check	Greatly reduce branch overbooking.
Static spreadsheets create visual complexity and obscure real-time operational insights.	A centralized web interface offering real-time visibility, filtered views, and AI decision trace logs.	Converge Hub	Complete migration from Excel to the web dashboard as the single source of truth.
Without automated disaster recovery, a system failure or database corruption could permanently erase all multi-branch schedules and halt business operations.	Generates scheduled, asynchronous database snapshots to provide secure, automated recovery points	Auto-Backup	Maintain business continuity by enabling rapid system restoration from automated daily snapshots.

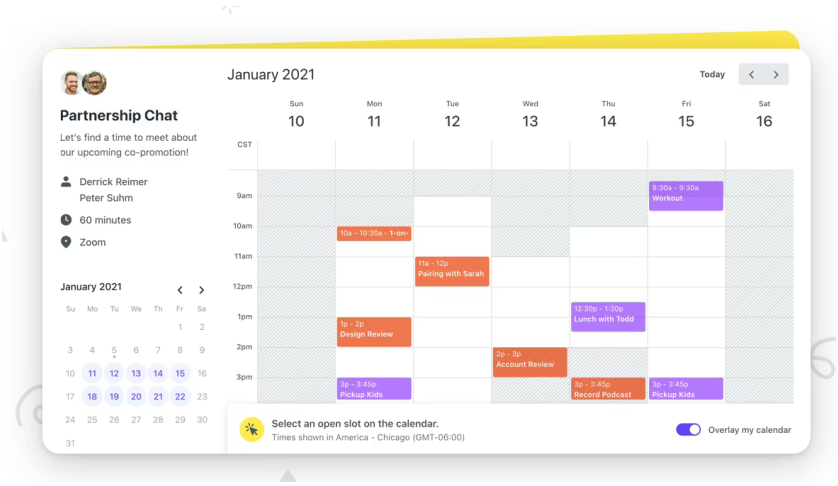
Chapter Two | Literature Review

2.1 Business Review / Software Review

Calendly

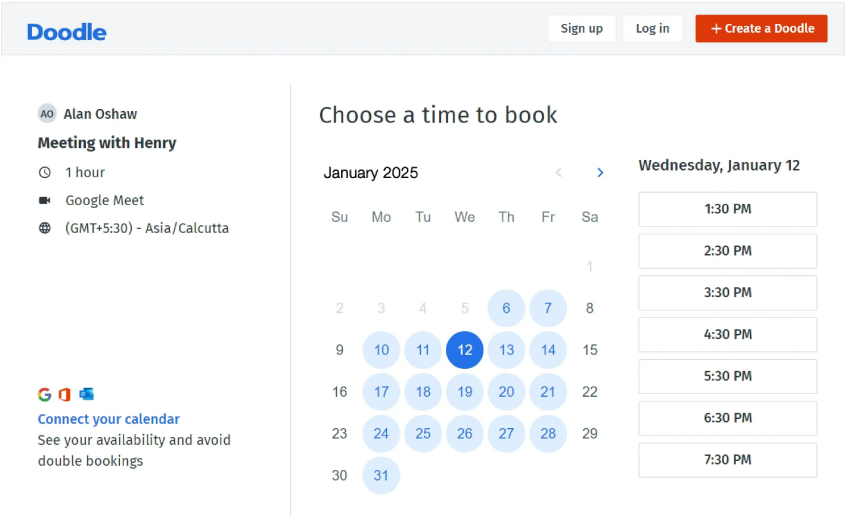
Calendly[2] is an automated scheduling platform that eliminates back-and-forth emails by allowing users to share their availability links, enabling invitees to instantly book open time slots.

Savvycal



Savvycal[3] is a collaborative scheduling tool featuring an interactive calendar overlay, which allows recipients to lay their own calendar over the sender's schedule to easily find mutual availability.

Doodle



Doodle[4] is a group scheduling platform that utilizes a polling system, where an organizer proposes multiple time options and participants vote on their availability to determine the best consensus time for a meeting.

Comparative Table

Feature	Calendly	Savvycal	Doodle	Converge
Smart Booking System	Partial — Link-Based	Partial — Overlay-Based	Partial — Consensus-Based	Auto-Pick
Commute-Aware Scheduling	Partial — Fixed Buffers	Partial — Fixed Buffers	No — Not Supported	Dynamic Buffers
Automatic Room Management	Partial — Text-Only	No — Not Supported	No — Not Supported	Auto-allocates
Centralized Schedule Dashboard	NO — User-Centric	No — User-Centric	No — Event-Centric	Business - Centric

2.2 Architectural Foundations & Prior Art

Because generic SaaS tools fail to process spatial-temporal routing and multi-variable constraints, Converge abandons standard scheduling APIs. Instead, the system architecture is built upon established Operations Research and Artificial Intelligence methodologies to solve these specific logistical bottlenecks.

2.2.1 Constraint Satisfaction Modeling for Complex Timetabling

Educational timetabling is a highly complex scheduling problem. As the number of instructors, branches, and time slots increases, the possible scheduling combinations grow exponentially, making it very difficult to process[1].

Rather than relying on a generalized model, Converge defines a specific set of rules tailored to multi-branch tutoring centers. The AI engine enforces strict physical realities (Hard Constraints)—such as ensuring an instructor is scheduled only once at any given time, and verifying that the total number of concurrent bookings does not exceed the fixed physical capacity of a branch.

As established in timetabling literature[5], the objective of the Converge engine is not to brute-force a single "perfect" schedule, as that requires too much computing power at scale. Instead, the system focuses on generating valid, conflict-free schedules that strictly follow all hard rules, while doing its best to satisfy administrative preferences (Soft Constraints) using an automated scoring system.

2.2.2 Travel Buffer System

Generic SaaS scheduling applications are fundamentally ignorant of physical geography, relying entirely on simple temporal time-blocking. However, experience within the timetabling research community dictates that every single institution operates under unique spatial and operational constraints, rendering off-the-shelf software insufficient for specialized educational logistics[6].

A critical requirement of the Converge architecture is safely dispatching personnel between physical branches without relying on fragile, third-party geographic location APIs. As established by state-of-the-art timetabling benchmarks like the ITC-2019 formulation[6], accounting for specific travel times between physical locations is a foundational requirement for generating valid, real-world schedules. Rather than introducing the unacceptable latency and system dependency of live spatial-routing APIs, Converge addresses this requirement by implementing a deterministic Inter-Branch Transit Matrix.

By calculating commute buffers based on an admin-defined, pre-configured travel duration matrix between branch pairs, the system automatically injects the required transit time into the scheduling engine. This approach instantly filters out physically impossible transit routes, keeping the system fast and preventing human errors caused by missed travel buffers.

2.2.3 Heuristic Search and Interactive Explainable AI (XAI)

When exact matches for student requests fail due to spatial-temporal or resource limitations, the system must calculate optimized alternatives. Because the search space is massive, the architecture implements Heuristic Search methodologies to rapidly score and return the next highest-scoring schedule vectors[1].

Furthermore, fully automated ”push-button” timetabling systems often fail in practice because human administrators require the flexibility to negotiate soft constraints directly with clients. Therefore, following Schaerf’s findings on automated timetabling[1], Converge is designed as an Interactive Decision Support System. The Symbolic AI engine processes the heavy spatial-temporal mathematics and exposes its logical trade-offs to the frontend via an Explainable AI (XAI) decision trace. This interactive architecture empowers administrative staff to manually finalize sales-driven booking decisions while relying on the backend engine to strictly enforce physical routing and capacity boundaries.

2.3 Tool and Technology Review

2.3.1 Frontend Language

We chose **TypeScript**[7] over JavaScript as the frontend language because its compile-time error checking prevents structural logic failures in our multi-branch scheduling. In making this choice, we accept the tradeoff of a slower initial prototyping phase due to strict type definition requirements.

Aspect / Language	TypeScript	JavaScript	Dart
Typing System	Static Typing	Dynamic Typing	Static Typing
Data Structure	Explicit Contracts	Flexible Objects	Classed Based Model
Development Tooling	Strong IDE support	Basic inference	Strong Tooling
Trade Off	High strictness overhead	Risk of runtime errors	Smaller Web Ecosystem

2.3.2 Frontend Framework

We chose **VueJS**[8] over React because its lower learning curve and built-in tooling allow us to focus on building features rather than configuring the project. In making this choice, we accept the tradeoff of a smaller ecosystem.

Aspect / Framework	VueJS	ReactJS	Angular
Component Structure	Single File Component	JSX	Component + Template
State Management	Proxy-based	Hooks	Services + RxJS
Ecosystem	Small ecosystem (official tools)	Massive ecosystem	Large enterprise ecosystem
Trade Off	Smaller community	Steep learning curve	Heavy framework, verbose

2.3.3 Backend Language

We chose **Golang**[9] over Java and NodeJS as the backend language because its compiled performance and native concurrency model perfectly balance raw execution speed with a low memory footprint. In making this choice, we accept the tradeoff of Go's verbose syntax and manual error handling, which requires writing more boilerplate code than Node.js.

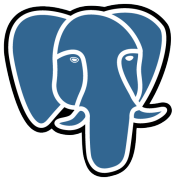
Aspect / Language	Golang	Java	NodeJS
Concurrency Model	Native	Thread-Based	Single-Threaded
Execution Model	Native Binary	JVM Bytecode	JIT Interpreted
Architecture	Minimalist & Explicit	Strict OOP	Highly Dynamic
Trade Off	Verbose syntax	High memory usage	Potential bottlenecks

2.3.4 Backend Framework

We chose the **Gin**[10] framework over Go's native standard library (`net/http`) because its built-in routing and middleware capabilities significantly accelerate the development of our REST API. In making this choice, we accept the tradeoffs of framework lock-in and a slight memory overhead.

Aspect / Framework	Gin	net/http	Fiber
Routing	Advanced	Basic	Express-style, fast
Middleware Management	Built-in	Manual	Built-in
Data Binding & Validation	Automated	Manual	Limited
Ecosystem	Mature	Minimal	Growing
Trade Off	Slight overhead & lock-in	More boilerplate	Less standard, smaller ecosystem

2.3.5 Database



We chose **PostgreSQL**[11] as the database because its native support for range data types is perfectly suited for a scheduling system. In making this choice, we accept the tradeoff of a steeper learning curve and slightly more configuration overhead.

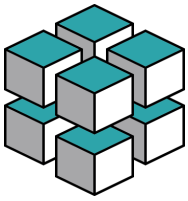
Aspect / Database	PostgreSQL	MySQL	MongoDB
Data Model	Relational	Relational	Document
Querying Complex Data	Advanced	Standard	Aggregation-heavy
Data Type Support	Range Types	Basic Types	Flexible Documents
Trade Off	Steeper learning curve	Limited advanced types	Risk of inconsistency

2.3.6 Architectural Pattern

We chose **Hexagonal Architecture**[12] as the backend architectural pattern because it keeps the core scheduling domain portable and testable in isolation from the web framework. In making this choice, we accept the tradeoff of increased initial complexity, requiring more folders and interfaces upfront.

Aspect / Architect	Modular Monolith	Microservice	SOA
System Complexity	Moderate	High	Moderate–High
Domain Modeling	Well-structured	Distributed	Service-based
Operational Overhead	Low	High	Moderate
Deployment	Single deployable unit	Independently deployable services	Multiple services
Trade Off	Requires discipline	High infrastructure cost	Less flexible than microservices, shared dependencies

2.3.7 Deployment Architecture



We chose a **Modular Monolith**[13] over both Microservices and a standard Monolith to provide a safe evolution path for the system. This approach allows us to properly model the system using Domain-Driven Design (DDD) principles without the immediate operational complexity of a distributed network. In making this choice, we accept the tradeoff of relying on strict developer code discipline to enforce module boundaries, rather than relying on physical infrastructure separation.

Aspect / Architect	Modular Monolith	Microservice	Monolith
System Complexity	Moderate	High	Low
Domain Modeling	Well-structured	Distributed	Unstructured
Operational Overhead	Low	High	Low
Trade Off	Requires discipline	High infrastructure cost	Technical debt risk

2.3.8 Containerization

We chose **Docker**[14] for containerization over traditional bare-metal because it keeps the setup environment consistency across development. By packaging the application and its dependencies into a single image, we eliminate environment-specific bugs. In making this choice, we accept the tradeoff of an increased learning curve, as the entire development team must understand Docker.

Aspect / Strategy	Docker	Podman	Bare Metal
Environment Consistency	High	High	Variable
Resource Isolation	High	High	Low
Deployment Speed	Fast	Fast	Slow
Resource Overhead	Low	Low	None
Security Model	Daemon-based	Daemonless	Host-dependent
Trade Off	Requires daemon, slight overhead	Less mature ecosystem	Environment inconsistency

Chapter Three | Project Plan

3.1 Objectives

The primary objective of this project is to successfully design, develop, and deploy the Converge scheduling system. By the end of the development lifecycle, the project aims to achieve the following measurable goals:

1. **Automate the Booking Workflow:** To drastically reduce the estimated 1 to 2 hours of daily administrative overhead by replacing manual spreadsheet filtering and repetitive messaging with an automated matching engine.
2. **Eliminate Scheduling Conflicts:** To implement a commute-aware algorithm and automated room allocation system that is built to stop double-bookings and enforces cross-branch travel buffers for instructors.
3. **Enhance Operational Visibility:** To deliver a centralized, interactive web dashboard that provides business owners and administrators with a single, real-time source of truth for branch operations.
4. **Implement a Scalable Architecture:** To engineer a robust backend utilizing a Modular Monolith and Hexagonal Architecture, ensuring the core scheduling domain is testable, highly maintainable, and prepared for future organizational scaling.

3.2 Type of System Users and the Users' Characteristics

User Type	Description
Admin	the operators who manage course registrations and schedule bookings based on the incoming requests from parents.
Parent	the users who provide the subject that they want their child to learn to the admin.

User Journey

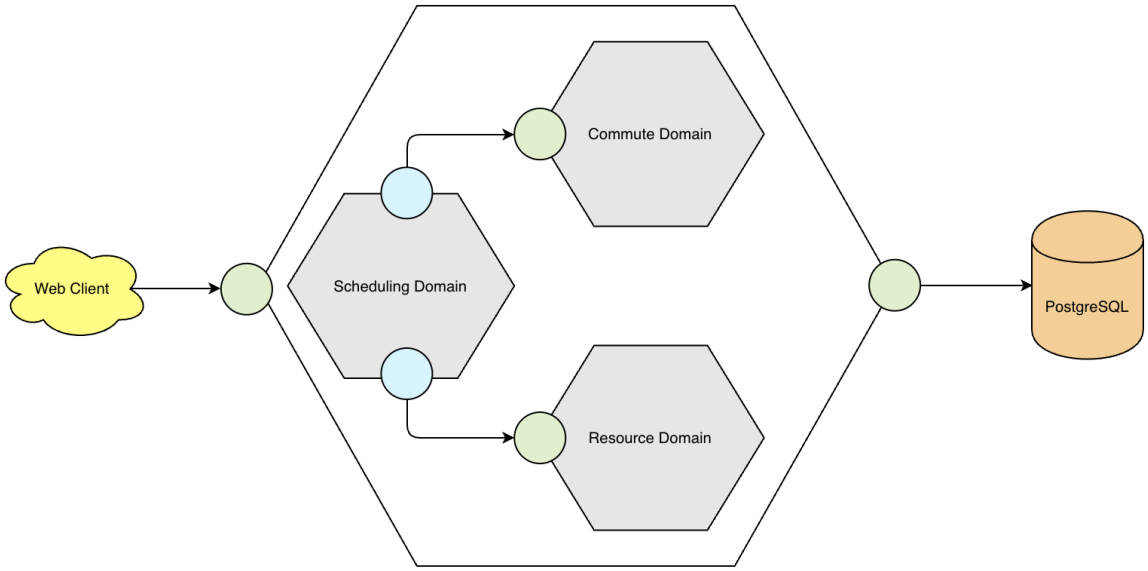
- **As is:** A parent contacts the admin to book a class. The admin has to manually search through a cluttered Excel spreadsheet to find an open time slot. If the time works, the admin then has to manually calculate the teacher's travel time between branches and act as a middleman, sending multiple LINE messages back and forth to confirm with everyone. Even after all this manual work, they might arrive at the branch only to find no physical room is actually available.
- **To be:** A parent contacts the admin to book a class. The admin simply inputs the request into Converge. The system instantly auto-matches the available teacher and student , automatically adds the travel time for the teacher , and permanently locks in a physical room. The booking is confirmed accurately in seconds, completely cutting out the messy spreadsheets and group chats.

3.3 Acronyms and Definitions

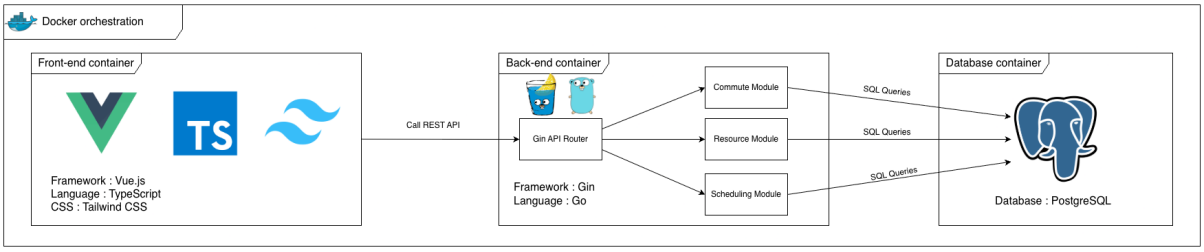
Acronyms	Description
Admin	The operators who manage course registrations and schedule bookings based on the incoming requests from parents.
API	**Application Programming Interface**[15]; used in the context of REST API for communication between the frontend and backend.
Commute Buffer	An automatically calculated travel time inserted between an instructor's consecutive classes at different physical branches.
DDD	**Domain-Driven Design**[16]; a software design approach focused on modeling the system based on the core business domain.
Hexagonal Architecture	A backend architectural pattern that keeps the core scheduling domain portable and testable in isolation from the web framework and database.
Modular Monolith	A deployment architecture where the system is deployed as a single unit but internally structured into well-defined modules.
Parent	The users who provide the subject that they want their child to learn to the admin.
REST	Representational State Transfer[17]; the architectural style used for designing the system's web API.
SaaS	**Software as a Service**[18]; a software distribution model mentioned as out-of-scope for this project.
SDLC	Software Development Life Cycle; the framework used for planning, creating, testing, and deploying the software.
XP	Extreme Programming; an Agile software development methodology focused on software quality, combined with Kanban for this project.

3.4 System Architecture

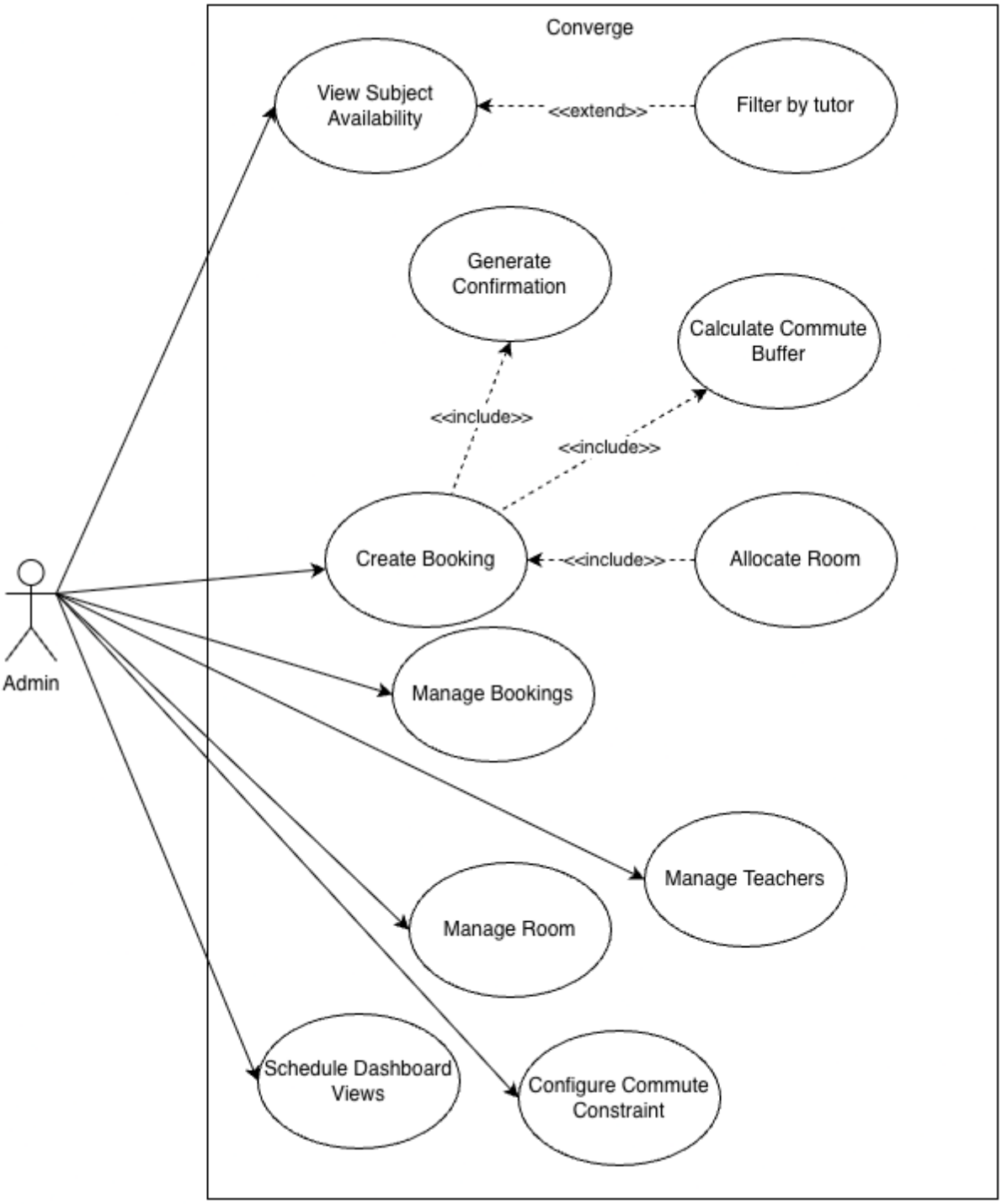
1.Hexagonal Architecture



2.Modular Monolith Deployment Architecture



3.5 Use Case Diagram



3.6 Product Features

3.6.1 Automated Data Ingestion Form

Description: An account-less, structured web form where teachers select their identity to submit their weekly availability.
Actor: Instructors (Teachers)
Details:

- **Structured Input:**
  - Replaces unstructured LINE chat paragraphs with a strict data validation layer.
- **Data Normalization:**
  - Ensures all instructor availability is instantly normalized into discrete binary matrices before entering the database, eliminating the need for administrative manual data entry.

3.6.2 Constraint Satisfaction AI Engine

Description: Evaluates hard limits (instructor availability, branch capacity) to output valid, rule-compliant schedules, rather than attempting to brute-force a single perfect solution.
Actor: Admin
Details:

- **Request Input:**
  - Admin inputs the requested subject and target branch. Specific time and instructor preferences are optional, allowing the system to suggest all possibilities.
- **Constraint-Based Filtering:**
  - Evaluates hard limits (instructor availability, branch capacity) to mathematically guarantee conflict-free schedules.
- **Heuristic Auto-Suggestion:**
  - If exact requests fail, the engine uses heuristic scoring to instantly suggest the next best optimal alternatives in real-time.

3.6.3 Travel Buffer System

Description: A configuration-driven module that automatically enforces deterministic travel buffers when an instructor changes branch locations.
Actor: System / Admin
Details:

- **Buffer Configuration:**
  - Provides an interface for administrators to define custom travel durations between specific branch pairs. Any pair without a custom value automatically falls back to a system-wide default.

- **Deterministic Injection:**
  - Automatically shifts an instructor's available start time based on the configured durations, instantly pruning physically impossible transit paths from the search space without relying on external APIs.
- **Visual Buffer Integrity:**
  - The master schedule explicitly displays these calculated commutes as "Travel Needed" to ensure operational clarity for administrators.

3.6.4 Branch Capacity Monitoring

Description: A constraint module that monitors total concurrent classes against the fixed physical capacity of each branch to prevent overbooking.
Actor: System
Details:

- **Capacity Definition:**
  - Administrators define a fixed integer representing the total number of usable spaces at each branch. The system tracks this total threshold rather than assigning specific named rooms.
- **Concurrent Tracking:**
  - Continuously calculates the total number of overlapping bookings at a given branch during a requested time slot.
- **Capacity Feedback & Prevention:**
  - Explicitly flags time slots as "At Capacity" if a new booking would exceed the physical threshold, systematically preventing branch overbooking before a schedule is confirmed.

3.6.5 Interactive XAI Decision Dashboard

Description: A centralized web interface that replaces static spreadsheets, acting as the administration's single source of truth and Explainable AI (XAI) hub.
Actor: Admin
Details:

- **Instructor Booking List:**
  - Displays a structured, real-time list of all teachers and their complete set of booked classes across all dates.
- **Decision Trace Logs:**
  - Exposes the heuristic scoring of the backend engine, visually explaining why certain times are blocked (e.g., commute violations or capacity limits) to aid staff in client negotiations.

- **Operational Filtering:**
  - Enables instant filtering by specific instructors, branches, and dates without the visual clutter of standard spreadsheet software.

3.6.6 Automated Disaster Recovery

Description: A backend pipeline that ensures business continuity and enterprise data sovereignty.
Actor: System
Details:

- **Asynchronous Snapshots:**
  - Generates scheduled, automated database snapshots without interrupting real-time scheduling operations.
- **Rapid Restoration:**
  - Provides secure recovery points to quickly restore multi-branch schedules and maintain business operations in the event of hardware failure or database corruption.

3.7 SDLC

Based on our project scope and team size, we decided to use Agile methodologies. It is perfectly suited for a small, remote team like ours that is still in the learning phase and operating under short deadlines. Agile provides the exact flexibility we need to adapt and succeed.

After confirming Agile as our foundation, we chose to implement a Kanban + XP hybrid style. Kanban tells us what to work on next, while XP tells us how to write it well. Neither alone is enough, but together, they cover everything a 2-person remote team needs to deliver high-quality software.

Furthermore, we realized that a Continuous Flow (Kanban) fits our dynamics much better than fixed 2-week sprints. Since our team members have varying schedules and features naturally finish at different speeds, continuous flow allows us to pull work asynchronously without blocking each other's progress.

3.8 Limits

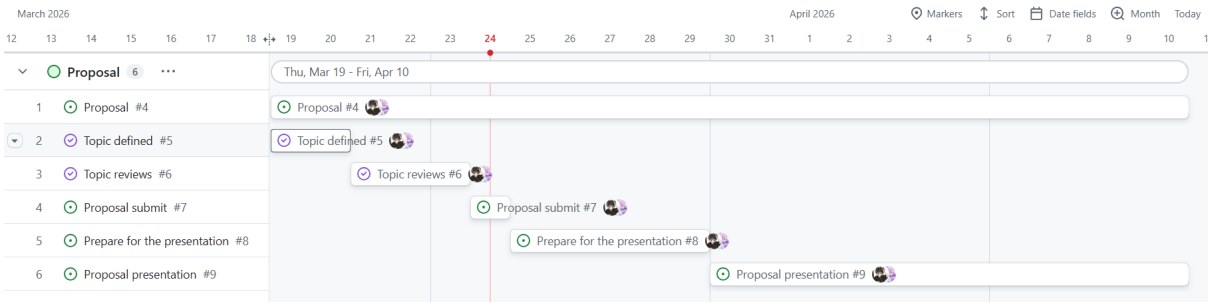
1. **Administrative Access Only:** The system is designed as a back-office tool for center managers. It does not include a student-facing portal, tutor login accounts, or self-service booking interfaces for parents.
2. **Non-Financial Scope:** The application does not handle any monetary transactions, including tuition fee collection, payroll processing for tutors, or tax invoicing.
3. **No Automated External Notifications:** While the system calculates the optimal schedule, it does not send automated Email, or LINE messages. All communication with parents and tutors remains a manual process handled by the admin.

4. **Fixed Time Commute Logic:** Commute buffers are calculated based on a configurable branch-pair matrix rather than real-time traffic data, eliminating dependency on external APIs. A default of 30 minutes applies to any pair without a custom value defined.
5. **Single Organization Focus:** The architecture is designed for a single tutoring business with multiple physical branches. it is not a multi-tenant SaaS for multiple independent companies.

3.9 Milestones

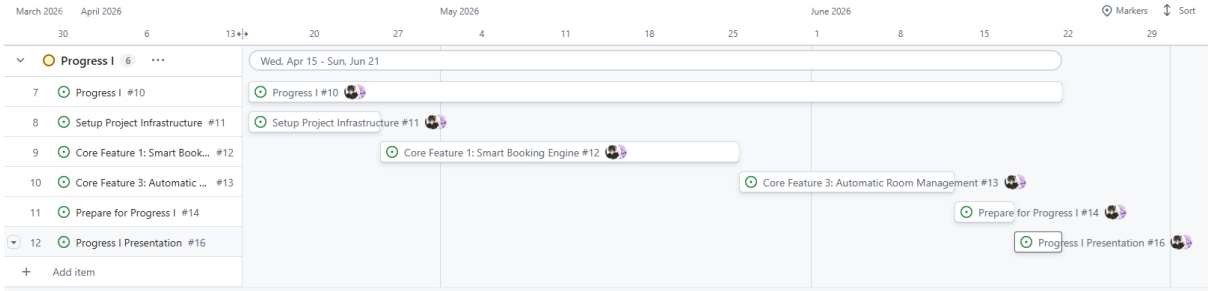
Phase 1: Proposal — (30 March 2026 - 10 April 2026)

Task	Start Date	End Date	Duration	Responsibility
Project Proposal	19 March 2026	10 April 2026	23 days	PT, SP
Topic defined	19 March 2026	10 April 2026	23 days	PT, SP
Topic reviews	21 March 2026	23 March 2026	3 days	PT, SP
Proposal submit	24 March 2026	24 March 2026	1 days	PT, SP
Prepare for the presentation	25 March 2026	29 March 2026	5 days	PT, SP
Proposal presentation	30 March 2026	10 April 2026	12 days	PT, SP



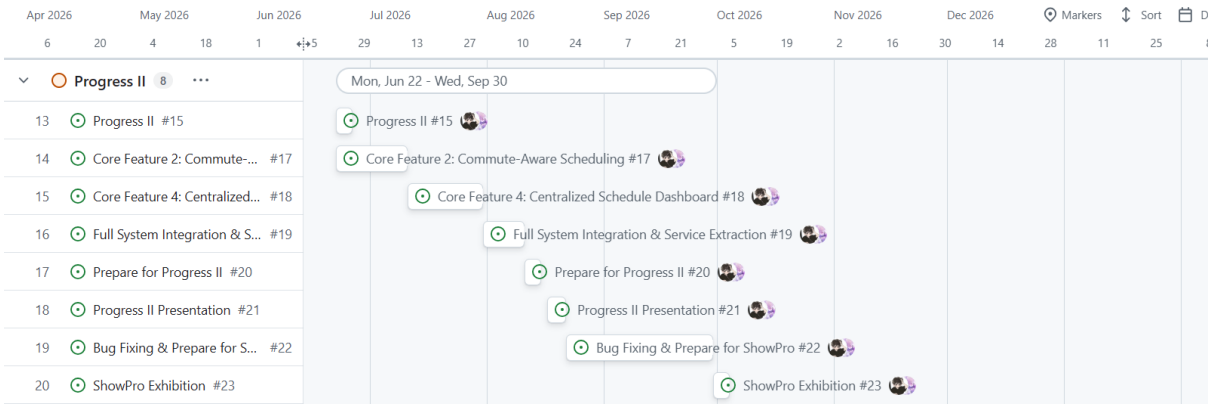
Phase 2: Progress I — (15 April 2026 - 21 June 2026 2026)

Task	Start Date	End Date	Duration	Responsibility
Setup Project Infrastructure	15 April 2026	25 April 2026	11 days	PT, SP
Core Feature 1: Smart Booking Engine	26 April 2026	25 May 2026	14 days	PT, SP
Core Feature 3: Automatic Room Management	26 May 2026	12 June 2026	18 days	PT, SP
Prepare for Progress I	13 June 2026	17 June 2026	5 days	PT, SP
Progress I Presentation	18 June 2026	21 June 2026	4 days	PT, SP



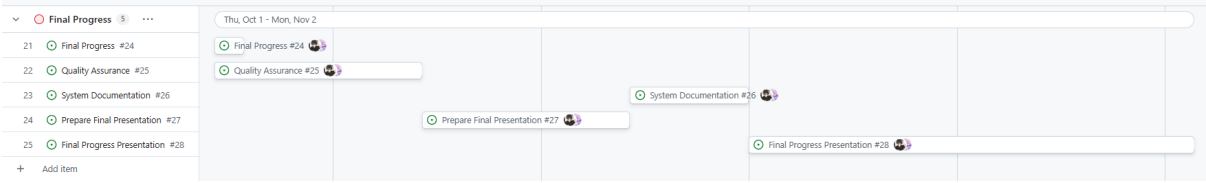
Phase 3: Progress II — (22 June 2026 - 30 September 2026)

Task	Start Date	End Date	Duration	Responsibility
Core Feature 2: Commute-Aware Scheduling	22 June 2026	10 July 2026	19 days	PT, SP
Core Feature 4: Centralized Schedule Dashboard	11 July 2026	30 July 2026	20 days	PT, SP
Full System Integration & Service Extraction	31 July 2026	10 August 2026	11 days	PT, SP
Prepare for Progress II	11 August 2026	14 August 2026	4 days	PT, SP
Progress II Presentation	17 August 2026	21 August 2026	5 days	PT, SP
Bug Fixing & Prepare for ShowPro	22 August 2026	29 September 2026	39 days	PT, SP
ShowPro Exhibition	30 September 2026	30 September 2026	1 days	PT, SP



Phase 4: Final Progress — (1 October 2026 - 2 November 2026)

Task	Start Date	End Date	Duration	Responsibility
Quality Assurance	1 October 2026	7 October 2026	7 days	PT, SP
System Documentation	8 October 2026	14 October 2026	7 days	PT, SP
Prepare Final Presentation	15 October 2026	18 October 2026	4 days	PT, SP
Final Progress Presentation	19 October 2026	2 November 2026	15 days	PT, SP



	Pct of task complete (%)	
	Progress I	**Progress II**
Feature 1: Smart Booking Engine	0%	0%
1.1 Request Input 1.2 Constraint-Based Filtering 1.3 Auto Suggestion 1.4 Booking Confirmation		
Feature 2: Commute Aware Scheduling	0%	0%
2.1 Travel Buffer Enforcement 2.2 Deterministic Logic 2.3 Visual Buffer Integrity		
Feature 3: Automatic Room Allocation	0%	0%
3.1 Room Management 3.2 Branch Travel Configuration 3.3 Standardized Allocation 3.4 Capacity Feedback		
Feature 4: Centralized Schedule Dashboard	0%	0%
4.1 Instructor Booking List 4.2 Alternative Visualizations 4.3 Operational Filtering & Management 4.3 Commute Time Configuration		

Chapter Four | References

[1] Andrea Schaerf. A survey of automated timetabling. *Artificial Intelligence Review*, 13(2):87–127, 1999.

[2] Calendly official website [online, accessed 18-03-2026] https://calendly.com.

[3] Savvycal official website [online, accessed 18-03-2026] https://savvycal.com.

[4] Doodle official website [online, accessed 18-03-2026] https://doodle.com.

[5] Tarek El-Sakka. University course timetable using constraint satisfaction and optimization. *International Journal of Computing Academic Research (IJCAR)*, 4(3):83–95, 2015.

[6] Sara Ceschia, Luca Di Gaspero, and Andrea Schaerf. Educational timetabling: Problems, benchmarks, and state-of-the-art results. *arXiv preprint arXiv:2201.07525*, 2022.

[7] Typescript official website [online, accessed 18-03-2026] https://typescriptlang.org.

[8] Vue official website [online, accessed 18-03-2026] https://vuejs.org.

[9] Golang official website [online, accessed 18-03-2026] https://go.dev.

[10] Gin official website [online, accessed 18-03-2026] https://gin-gonic.com.

[11] Postgressql official website [online, accessed 18-03-2026] https://www.postgresql.org.

[12] Hexagonal website by aws [online, accessed 18-03-2026] https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html.

[13] Modular monolith vs microservice website [online, accessed 18-03-2026] https://www.milanjovanovic.tech/blog/what-is-a-modular-monolith.

[14] Docker official website [online, accessed 18-03-2026] https://www.docker.com.

[15] Api website [online, accessed 18-03-2026] https://aws.amazon.com/th/what-is/api/.

[16] Domain-driven design website [online, accessed 18-03-2026] https://medium.com/devspree/domain-driven-design-934c495e7e0b.

[17] Rest website [online, accessed 18-03-2026] https://developer.mozilla.org/en-US/docs/Glossary/REST.

[18] Saas website [online, accessed 18-03-2026] https://aws.amazon.com/th/what-is/saas/.